DigitalTwin for MyPalletizer 260 M5 Unity Version

Inhaltsverzeichnis

1. Abstract . . . 2
2. Introduction and Goals . . . 3
3. Architecture Constraints . . . 5
4. Context and Scope . . . 7
5. Solution Strategy . . . 13
6. Building Block View . . . 16
7. Runtime View . . . 24
8. Deployment View . . . 28
9. Cross-cutting Concepts . . . 30
10. Architecture Decisions . . . 33
11. Quality Requirements . . . 35
12. Risks and Technical Debts . . . 37
13. Glossary . . . 38

1. Abstract

Dieses Dokument beschreibt die Architektur des digitalen Zwillings für den MyPalletizer 260 M5. Das System ermöglicht die hybride Steuerung von Hardware und Simulation über eine einheitliche Python-Schnittstelle. Es dient primär Ausbildungszwecken an der Hochschule Luzern (HSLU).

2. Introduction and Goals

Das Projekt **DigitalTwin for MyPalletizer 260 M5** hat zum Ziel, eine realitätsnahe und performante Simulationsumgebung für den MyPalletizer-Roboterarm bereitzustellen.

Durch eine einheitliche Programmierschnittstelle für reale Hardware und Simulation kann derselbe Code in beiden Umgebungen verwendet werden. Dadurch wird das Lernen und Testen in der Robotik einfacher, sicherer und unabhängig vom Standort.

Dies ermöglicht es Studierenden und Robotik-Interessierten, die Funktionsweise des MyPalletizer besser zu verstehen, Bewegungsabläufe zu entwickeln und verschiedene Szenarien zu testen, ohne die Hardware zu gefährden oder vor Ort sein zu müssen.

Ein praktischer Hintergrund dafür ist, dass die Hochschule Luzern (HSLU) den MyPalletizer nur in einem Laborraum zur Verfügung stellt, der nicht jederzeit zugänglich ist. Zudem stehen lediglich zwei Roboter zur Verfügung, was die Nutzung zusätzlich einschränkt.

2.1. Functional Requirements

Das System erfüllt die folgenden funktionalen Anforderungen:

- **Hybride Steuerung** Umschaltung zwischen den Betriebsmodi `virtual` (Simulation), `real` (Hardware) und `both` (gleichzeitig) über eine zentrale Python-API. Dies wird im Kapitel Betriebsarten näher beschrieben.
- **Gelenksteuerung (Kinematik)**
  - Vorgabe von Gelenkwinkeln für alle 4 Achsen des Roboters
  - Unterstützung von synchronen Bewegungen (mehrere Gelenke gleichzeitig)
  - Einstellbare Bewegungsgeschwindigkeit
- **End-Effektor-Steuerung**
  - Steuerung eines Greifers (Öffnen/Schliessen)
  - Steuerung einer Vakuumpumpe
  - Auswahl des aktiven Werkzeugs (Greifer oder Pumpe)
- **Dynamische Szenariengestaltung**
  - Erstellen (Spawn) und Löschen von Objekten zur Laufzeit über Python
  - Unterstützung von einfachen 3D-Objekten (z. B. Box, Sphere, Cylinder)
  - Zuweisung von Position, Rotation, Skalierung und Farbe
  - Laden von vordefinierten Szenarien (Presets), siehe Kapitel Objekt- und Szenarienkonzept
- **Visuelles Feedback & Status**
  - Darstellung der Roboterbewegung in der Unity-Simulation
  - Steuerung der LED (RGB) am Roboter
  - Darstellung der Szene aus verschiedenen Kameraperspektiven in Unity

2.2. Quality Goals

Die Architektur orientiert sich an folgenden Qualitätszielen:

Ziel	Szenario	Maßnahme
Latenz	Ein Bewegungsbefehl soll möglichst schnell in der Simulation sichtbar sein (< 50 ms).	Verwendung von UDP für schnelle, verbindungslose Kommunikation.
Realitätsnähe	Die Simulation soll sich ähnlich wie der reale Roboter verhalten.	Begrenzung der Gelenkwinkel und Geschwindigkeiten entsprechend der Hardware.
Wartbarkeit	Neue Funktionen oder Tools sollen einfach ergänzt werden können.	Klare Trennung zwischen Robot- und World-Logik sowie modulare Klassenstruktur.
Portabilität	Das System soll auf verschiedenen Betriebssystemen laufen.	Einsatz von Python und Unity als plattformunabhängige Technologien.

2.3. Stakeholders

Stakeholder	Rolle	Interessen / Erwartungen
Studierende (HSLU)	Endnutzer	Einfache Nutzung, verständliche API, sichere Umgebung zum Testen ohne Risiko für die Hardware
Dozierende (Prof. Dr. Jana Koehler)	Auftraggeberin / Prüferin	Zuverlässiges Tool für Übungen und Projekte, einfache Reproduzierbarkeit
Robotik-Interessierte	Externe Nutzer	Gute Dokumentation, einfache Erweiterbarkeit, Open-Source Nutzung
Entwickler (Colin Felber)	Entwickler	Klare Struktur, wartbarer Code, saubere Dokumentation als Teil der Studienleistung

3. Architecture Constraints

Die Architektur des Digital Twins unterliegt verschiedenen technischen, organisatorischen und konventionellen Rahmenbedingungen, welche den Lösungsraum einschränken und die Umsetzung beeinflussen.

3.1. Technische Rahmenbedingungen

Randbedingung	Erläuterung
Programmiersprache	Die Steuerungsschicht ist in **Python (ab Version 3.12)** implementiert. Dies wurde gewählt, da Python von den meisten Nutzern bereits für die Entwicklung von Bewegungsabläufen und Szenarien verwendet wird und somit gut verständlich ist.
Simulations-Engine	Für die Visualisierung wird **Unity** verwendet. Dies erfordert eine Implementierung der Kommunikationslogik in **C#** sowie die Nutzung von Unity-spezifischen Komponenten.
Kommunikation (Hardware)	Die Ansteuerung des realen Roboters erfolgt über eine **serielle Schnittstelle (USB)** mit einer Baudrate von 115200.
Kommunikation (Simulation)	Die Kommunikation zwischen Python und Unity erfolgt über UDP mit JSON-basierten Nachrichten, welche im Kapitel Externe Schnittstellen näher beschrieben werden.
Prozess-Trennung	Steuerlogik (Python) und Simulation (Unity) laufen in getrennten Prozessen. Eine direkte Integration ist nicht vorgesehen, wodurch eine lose Kopplung entsteht.
Hardware-Plattform	Das System soll auf Standard-Laptops (Windows, Linux, macOS) lauffähig sein. Es wird keine spezialisierte Hardware vorausgesetzt.

3.2. Organisatorische Rahmenbedingungen

- **Open Source & Reproduzierbarkeit** Das Projekt wird über GitHub verwaltet. Alle Abhängigkeiten (z. B. `pymycobot`) müssen über Standard-Tools wie `pip` installierbar sein.
- **Zeitrahmen** Die Entwicklung erfolgt im Rahmen eines Semesters im Modul APPLAI.
- **Verfügbarkeit der Hardware** Die realen Roboter steht nur eingeschränkt Stückzahl im Labor zur Verfügung. Dies war ein zentraler Treiber für die Entwicklung der Simulation.
- **Nutzergruppe** Die Hauptzielgruppe sind Studierende und Robotik-Interessierte, die den MyPalletizer kennenlernen und Bewegungsabläufe entwickeln möchten. Dabei soll die Programmierung gleich aufgebaut sein, wie die Hersteller-API, sodass nicht andere Kenntnisse als die Hersteller-API notwendig sind.

3.3. Konventionen und Standards

- **Dokumentationsstandard** Die Architektur wird gemäss **ARC42** dokumentiert.
- **Coding Standards**
  - Python: Orientierung an **PEP 8**
  - C#: Orientierung an Unity-Konventionen
- **Kommunikationsprotokoll**
  - Verwendung eines JSON-basierten Protokolls
  - Versionierung der Nachrichten (aktuell Version 1)

3.4. Technische Annahmen

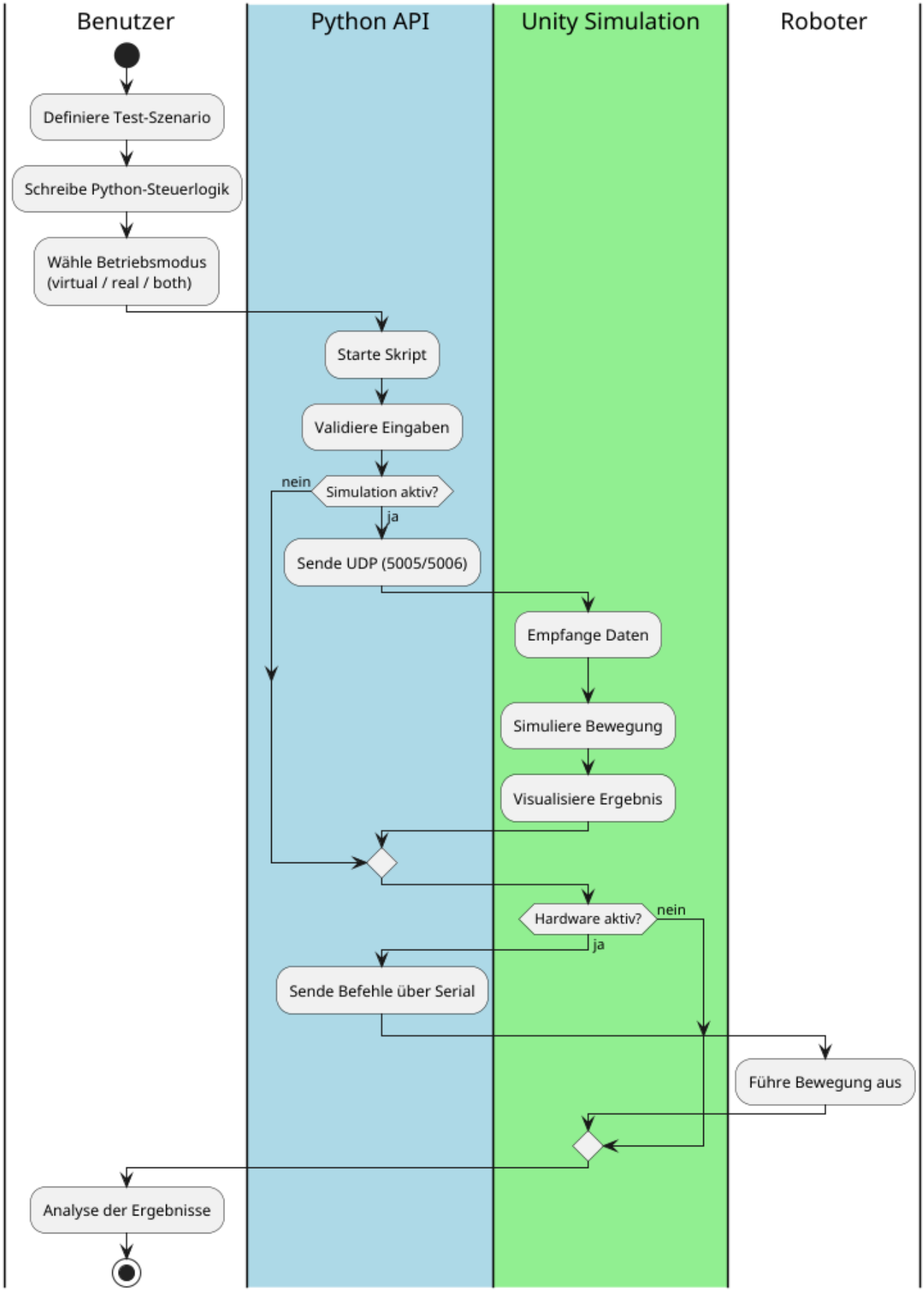
- Die UDP-Kommunikation erfolgt lokal (localhost) und ohne Paketverlust.
- Die Simulation muss keine vollständig physikalisch exakte Abbildung liefern, sondern ein plausibles Verhalten.
- Der Benutzer verfügt über grundlegende Python-Kenntnisse zur Nutzung der API.

4. Context and Scope

In diesem Kapitel wird das System gegenüber externen Akteuren abgegrenzt und der typische Ablauf der Nutzung beschrieben.

4.1. Business Context

Der Business Context beschreibt den Ablauf aus Sicht der Benutzer (Studierende / Robotik-Interessierte). Das folgende Diagramm zeigt eine typische Nutzung des Systems.

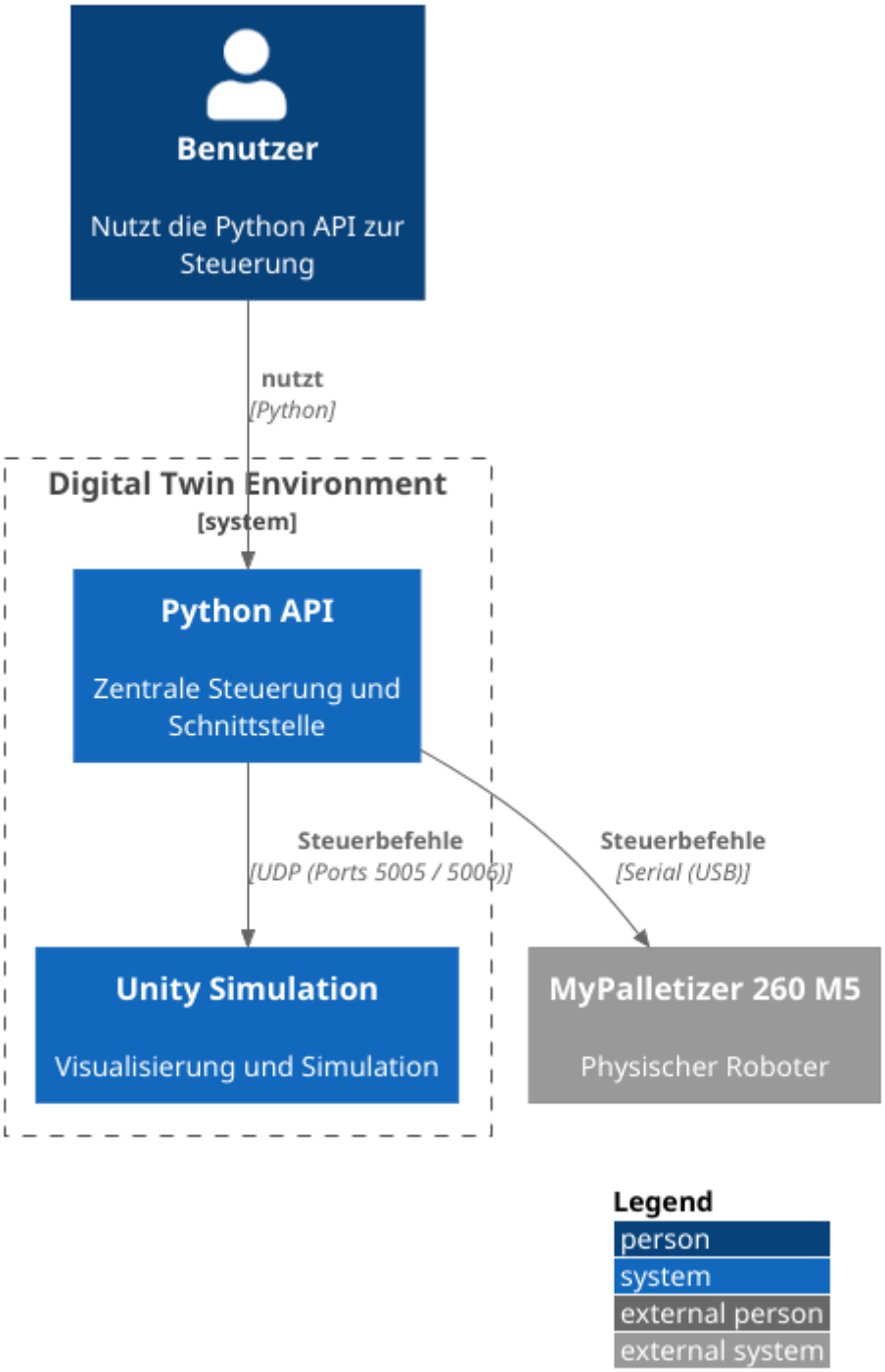


Akteur	Verantwortung im Prozess
Benutzer	Erstellt die Programmlogik und bewertet das Verhalten des Roboters in der Simulation oder auf der Hardware.

Akteur	Verantwortung im Prozess
Python API	Dient als zentrale Schnittstelle (Fassade) zum System. Sie vereinfacht die Nutzung, prüft Eingaben und verteilt Befehle je nach Betriebsmodus an Simulation und/oder Hardware.
Unity Simulation	Stellt die Bewegung des Roboters visuell dar und ermöglicht das Testen von Szenarien.
Hardware	Führt die Befehle auf dem realen Roboter aus.

4.2. Technical Context

Der technische Kontext zeigt die Systemgrenzen sowie die Kommunikationswege zwischen den einzelnen Komponenten.



4.2.1. Externe Schnittstellen

Schnittstelle	Protokoll	Zweck
Benutzer-API	Lokal (Python)	Stellt Funktionen wie `send_angles()`, `set_color()` oder `spawn()` zur Verfügung.
Hardware-Kanal	Serial (115200 Baud)	Überträgt Steuerbefehle an den realen Roboter.
Simulations-Kanal	UDP (JSON)	Überträgt Bewegungs- und Zustandsbefehle an die Simulation (Port 5005).
Welt-Kanal	UDP (JSON)	Überträgt Befehle zur Verwaltung der Simulationsumgebung (Port 5006).

Schnittstelle	Protokoll	Zweck
Trennung der Kanäle	-	Die Kommunikation zur Simulation ist in zwei Kanäle aufgeteilt. Der Roboterkanal ist für zeitkritische Bewegungsbefehle zuständig, während der Weltkanal für weniger häufige Aktionen wie das Erstellen oder Löschen von Objekten verwendet wird. Genauere Details für diesen Entscheid folgen im Kapitel Strategie zur Kommunikationstrennung.

4.2.2. Beispielhafte JSON-Nachrichten

Die Kommunikation zwischen Python und Unity erfolgt über strukturierte JSON-Nachrichten. Dabei wird zwischen Roboterbefehlen und Weltbefehlen unterschieden. Beide Nachrichtentypen verwenden eine gemeinsame Grundidee: Jede Nachricht enthält eine Protokollversion `v` sowie ein Feld `type`, das den Befehl eindeutig beschreibt.

Die konkreten Nachrichten werden auf Python-Seite durch das Modul `protocol` erzeugt. Dadurch bleibt der Aufbau der übertragenen Daten zentral definiert und konsistent.

Beispiel einer Nachricht für eine asynchrone Gelenkbewegung

```
{
  "v": 1,
  "type": "move_joints",
  "j1": 0.0,
  "j2": 20.0,
  "j3": -15.0,
  "j4": 10.0,
  "speed": 40
}
```

Diese Nachricht wird über den Roboterkanal an die Simulation übertragen und beschreibt den Soll-Zustand der vier Gelenke sowie die gewünschte Bewegungsgeschwindigkeit.

Beispiel einer Nachricht für die Werkzeugwahl

```
{
  "v": 1,
  "type": "set_end_effector",
  "tool": "gripper"
}
```

Diese Nachricht wählt das aktive Werkzeug des Roboters. Unterstützt werden insbesondere `gripper` und `pump`.

Beispiel einer Nachricht für das Erzeugen eines Objekts in der Simulationswelt

```
{
  "v": 1,
  "type": "spawn",
  "object_type": "box",
  "name": "box_1",
  "position": {
    "x": 0.2,
    "y": 0.0,
    "z": 0.1
  },
  "rotation": {
    "x": 0.0,
    "y": 0.0,
    "z": 0.0
  },
  "scale": {
    "x": 0.05,
    "y": 0.05,
    "z": 0.05
  },
  "color": {
    "r": 255,
    "g": 0,
    "b": 0
  }
}
```

Diese Nachricht wird über den Weltkanal übertragen und beschreibt ein neues Objekt, das in Unity erzeugt werden soll. Neben Typ und Name enthält sie auch Transformations- und Darstellungsparameter wie Position, Rotation, Skalierung und Farbe.

Die Beispiele zeigen bewusst nur einen kleinen Ausschnitt des Protokolls. Weitere Befehle wie `sync_move_joints`, `led`, `set_gripper_state`, `pump`, `delete` und `clear` folgen demselben Grundprinzip.

5. Solution Strategy

Die Lösungsstrategie basiert auf einer klaren Trennung zwischen der Steuerlogik (Python) und der visuellen bzw. physikalischen Darstellung (Unity).

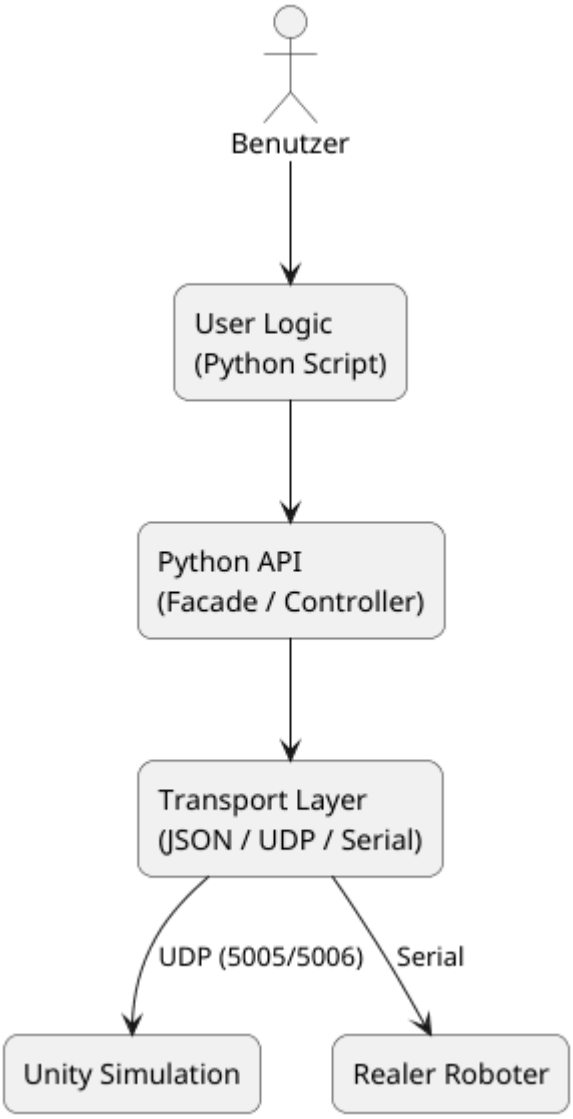
5.1. Architekturelle Leitprinzipien

1. **Einheitliche Abstraktion** Der Benutzer soll nicht unterscheiden müssen, ob er die Simulation oder die reale Hardware steuert. Alle modusspezifischen Unterschiede (`virtual`, `real`, `both`) werden intern im System gekapselt.
2. **Entkopplung durch Netzwerkschnittstellen** Durch die Verwendung von UDP laufen Steuerlogik (Python) und Simulation (Unity) unabhängig voneinander. Dadurch wird verhindert, dass Performance-Schwankungen der 3D-Engine die Steuerlogik beeinflussen.
3. **Zustandsbasierte Kommunikation** Jedes UDP-Paket enthält den vollständigen Soll-Zustand (z. B. alle Gelenkwinkel). Dadurch kann ein verlorenes Paket durch das nächste automatisch korrigiert werden.

5.2. Entscheidungen zur Kommunikation

- **Warum UDP statt TCP?** In diesem System ist eine geringe Latenz wichtiger als eine garantierte Zustellung. Ein verlorenes Paket ist unkritisch, da kurze Zeit später ein neues Paket mit aktuellen Daten gesendet wird. TCP würde durch Wiederholungen (Retransmissions) zu Verzögerungen führen.
- **Warum JSON?** JSON ist einfach zu implementieren und gut lesbar, was das Debugging erleichtert. Da die Datenmenge pro Nachricht klein ist, überwiegen diese Vorteile gegenüber binären Formaten.

5.3. Schichtenmodell



Schicht	Verantwortung
User Logic	Python-Skript des Benutzers. Enthält z. B. Bewegungslogik oder Szenarien.
API / Controller Layer	Zentrale Schnittstelle des Systems. Kapselt die Nutzung von `pymycobot`, validiert Eingaben und entscheidet, ob Befehle an Simulation, Hardware oder beide gesendet werden.
Transport Layer	Verantwortlich für die Übertragung der Daten (JSON, UDP, Serial).
Execution Layer	Führt die Befehle aus: Simulation in Unity oder reale Ausführung auf dem Roboter.

5.4. Strategie zur Kommunikationstrennung

Die Kommunikation zwischen Python und Unity erfolgt über zwei getrennte UDP-Ports. Dabei wird Port 5005 für Roboterbefehle wie Gelenkbewegungen, Werkzeugbefehle und LED-Steuerung verwendet, während Port 5006 für Befehle zur Verwaltung der Simulationswelt eingesetzt wird,

beispielsweise für `spawn`, `delete` oder `clear`. Diese Ports sind standardmässig festgelegt, können aber bei Bedarf angepasst werden.

Diese Trennung wurde bewusst gewählt, da beide Kommunikationsarten unterschiedliche Anforderungen haben. Roboterbefehle treten häufig auf und sind zeitkritisch, da Bewegungen möglichst flüssig und ohne Verzögerung in der Simulation sichtbar werden sollen. Befehle zur Weltverwaltung sind dagegen seltener und eher diskrete Einzelaktionen. Durch die Aufteilung auf zwei getrennte Kanäle wird verhindert, dass grössere oder unregelmässige Weltnachrichten die kontinuierliche Übertragung der Roboterbefehle beeinflussen.

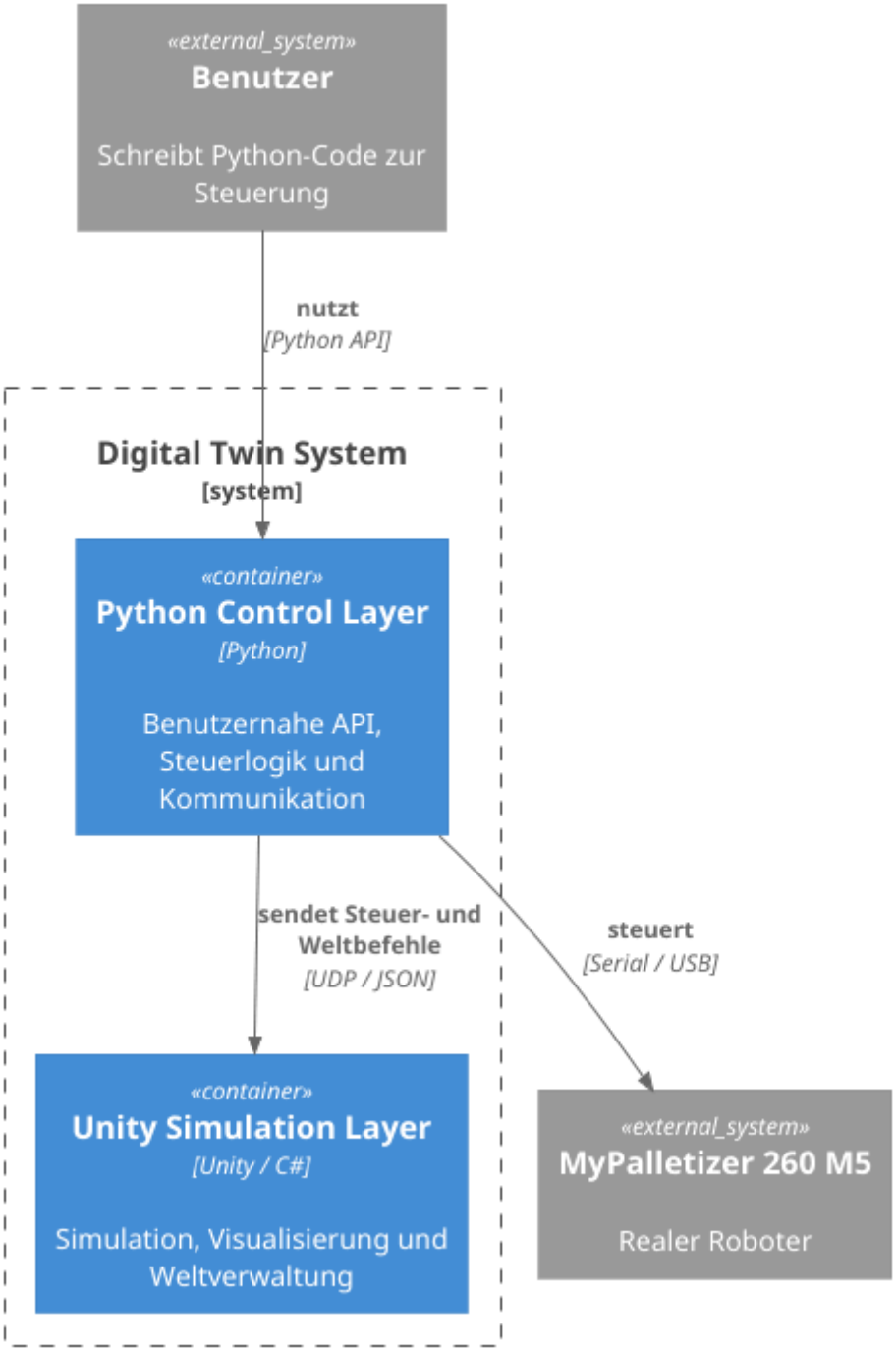
Zusätzlich verbessert diese Trennung die Struktur des Systems, da Robotersteuerung und Weltverwaltung auch konzeptionell als zwei unterschiedliche Aufgabenbereiche behandelt werden. Dadurch bleibt die Architektur klarer, leichter erweiterbar und einfacher zu warten.

6. Building Block View

Die Bausteinsicht beschreibt die statische Struktur des Systems und zeigt, wie es in einzelne Bausteine aufgeteilt ist. Dabei werden sowohl die Python-Seite als auch die Unity-Seite betrachtet. Zusätzlich wird gezeigt, wie der reale Roboter und die Herstellerbibliothek `pymycobot` eingebunden sind.

6.1. Whitebox Overall System (Level 1)

Auf der obersten Ebene besteht das System aus einer Python-basierten Steuerungs- und API-Schicht sowie einer Unity-basierten Simulationsschicht. Der reale Roboter ist ein externes System, das über die Python-Seite angebunden wird.



6.1.1. Enthaltene Bausteine

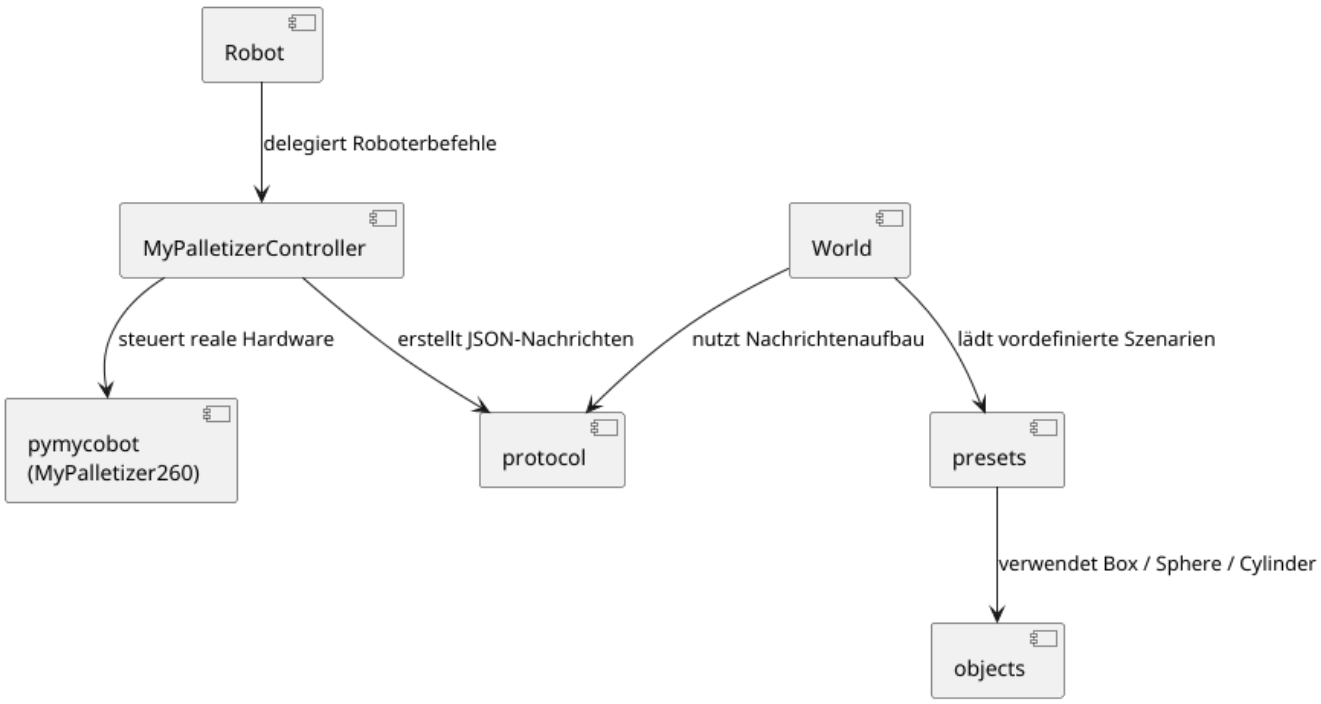
Baustein	Beschreibung
Python Control Layer	Stellt die zentrale Schnittstelle für den Benutzer bereit. Diese Schicht kapselt die internen Unterschiede zwischen Simulation, realer Hardware und kombiniertem Betrieb.
Unity Simulation Layer	Empfängt Befehle aus Python und setzt diese in simulierte Bewegungen, Werkzeugfunktionen und Änderungen der Simulationswelt um.
Realer Roboter (MyPalletizer 260 M5)	Führt Befehle auf der physischen Hardware aus. Die Ansteuerung erfolgt über die Herstellerbibliothek `pymycobot`.

6.2. Level 2

Auf Level 2 werden die internen Bausteine der Python- und Unity-Schicht genauer beschrieben.

6.2.1. Whitebox Python Control Layer

Die Python-Schicht ist so aufgebaut, dass der Benutzer mit einer einfachen API arbeitet, während die eigentliche Verteilung auf Simulation und Hardware intern erfolgt.



Beschreibung der Bausteine

Baustein	Beschreibung
Robot	Benutzernahe API für die Robotersteuerung. Diese Klasse kapselt die Nutzung des Controllers und bietet einfache Methoden wie zum Beispiel `send_angles()`, `sync_move_joints()`, `set_color()` `set_gripper_state()`, und `pump_on()`. Zudem validiert sie die Grundkonfiguration, z. B. den benötigten Serial-Port in den Modi `real` und `both`.

Baustein	Beschreibung
MyPalletizerController	Zentrale Steuerkomponente für den Roboter. Diese Klasse entscheidet anhand des Modus `virtual`, `real` oder `both`, ob ein Befehl an die Simulation, an den realen Roboter oder an beide weitergeleitet wird. Zusätzlich begrenzt sie Gelenkwinkel und Geschwindigkeit und verwaltet den aktuellen Simulationszustand der Gelenke.
World	API für die Verwaltung der Simulationswelt. Diese Klasse ist getrennt von der Robotersteuerung und bietet Funktionen wie zum Beipsiel `spawn()`, `delete()` und `load_preset()`.
protocol	Hilfsmodul zum Aufbau der JSON-Nachrichten. Es definiert sowohl Roboterbefehle wie zum Beispiel `move_joints`, `led` und `pump` als auch Weltbefehle wie `spawn` und `delet`.
presets	Enthält vordefinierte Szenarien für die Simulation, z. B. `empty`, `three_blocks`, `pick_and_place_basic` und `sorting_task`.
objects	Definiert die einfachen Weltobjekte `Box`, `Sphere` und `Cylinder`, welche durch die `World`-Klasse in die Simulation eingebracht werden können.
pymycobot	Externe Herstellerbibliothek zur Ansteuerung des realen MyPalletizer. Sie wird nicht direkt vom Benutzer verwendet, sondern ausschliesslich intern über den `MyPalletizerController`.

Architekturidee der Python-Schicht

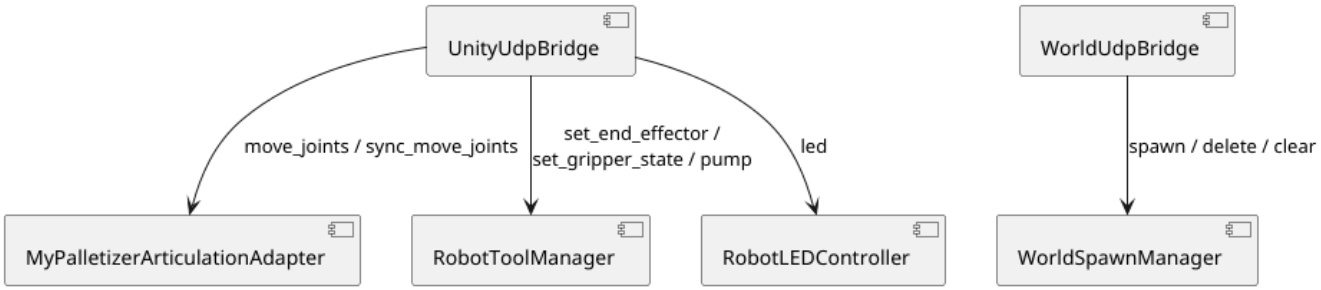
Wie schon im Kapitel <_strategie_zur_kommunikationstrennung, Strategie zur Kommunikationstrennung>> beschrieben, trennt die Python-Schicht und Unity-Schicht bewusst zwischen Robotersteuerung und Weltverwaltung. `Robot` und `World` bilden zwei eigenständige Zugriffspunkte für den Benutzer. Intern übernimmt `MyPalletizerController` die eigentliche Orchestrierung der Roboterbefehle, während `World` einen separaten Kommunikationsweg für Weltänderungen verwendet.

Dadurch ergibt sich eine klare Aufteilung:

- **Robot** für Bewegungen, Werkzeuge und LED
- **World** für Objekte und Szenarien
- **protocol** für die gemeinsame Nachrichtenstruktur
- **pymycobot** für die reale Hardwareanbindung

6.2.2. Whitebox Unity Simulation Layer

Die Unity-Schicht ist ebenfalls in zwei Hauptbereiche getrennt: Robotersteuerung und Weltverwaltung. Dadurch werden zeitkritische Roboterbefehle nicht mit Weltänderungen vermischt.



Beschreibung der Bausteine

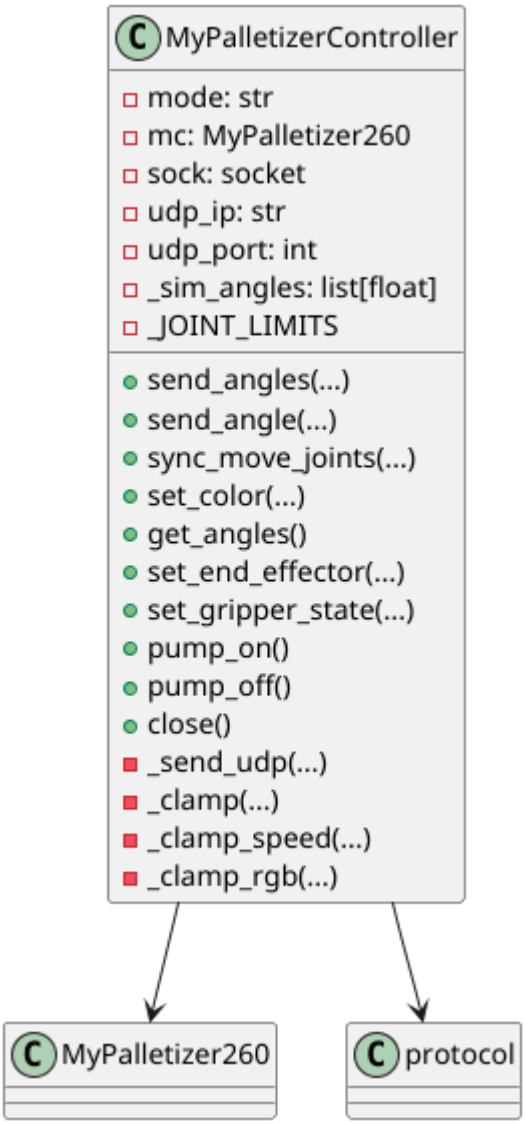
Baustein	Beschreibung
UnityUdpBridge	Empfängt Roboterbefehle über UDP auf Port `5005`. Die empfangenen JSON-Nachrichten werden zunächst in eine Queue gelegt und anschliessend im Unity-Hauptthread verarbeitet.
MyPalletizerArticulationAdapter	Übersetzt Gelenkbefehle in konkrete Bewegungen der vier simulierten Gelenke. Dabei werden sowohl asynchrone als auch synchrone Bewegungen unterstützt.
RobotToolManager	Verwaltet die aktiven Werkzeuge des Roboters innerhalb der Simulation. Diese Komponente reagiert auf Befehle zum Werkzeugwechsel sowie auf Greifer- und Pumpenbefehle.
RobotLEDController	Setzt empfangene RGB-Werte in die visuelle LED-Darstellung des Roboters um.
WorldUdpBridge	Empfängt Weltbefehle über UDP auf Port `5006`. Auch hier werden Nachrichten zunächst zwischengespeichert und anschliessend sicher im Unity-Hauptthread verarbeitet.
WorldSpawnManager	Verarbeitet die Weltbefehle `spawn`, `delete` und `clear`. Er erstellt Objekte, entfernt sie wieder oder setzt die Szene zurück.

6.3. Level 3

Auf Level 3 werden die wichtigsten internen Strukturen der Python- und Unity-Schicht detaillierter betrachtet.

6.3.1. Whitebox `MyPalletizerController`

Der `MyPalletizerController` ist der zentrale technische Kern der Python-Schicht. Er verbindet die Benutzer-API mit der Simulation und der realen Hardware.



Aufgaben des Controllers

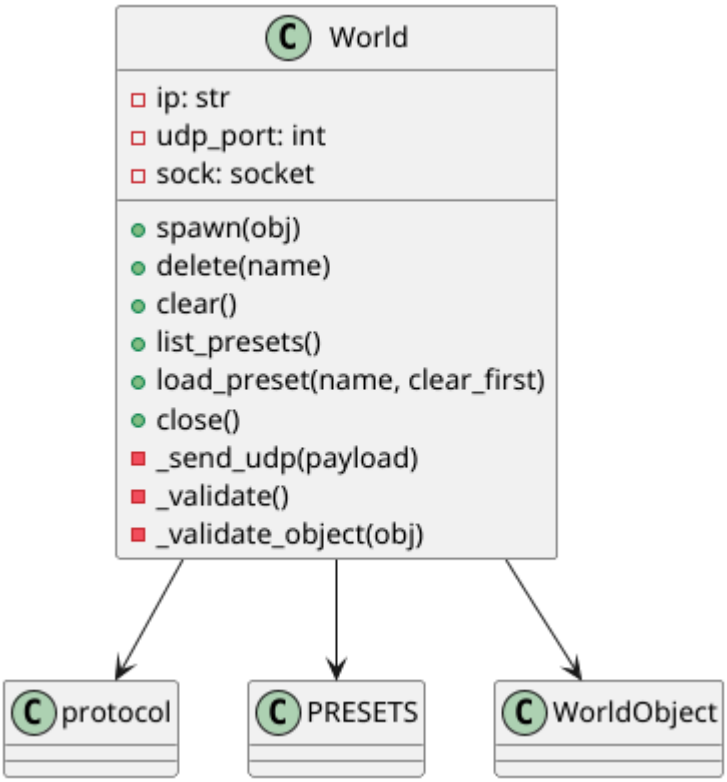
Der `MyPalletizerController` erfüllt mehrere Aufgaben gleichzeitig:

- **Modusabhängige Verteilung** Im Modus `virtual` werden Befehle nur per UDP an Unity gesendet. Im Modus `real` werden Befehle nur über `pymycobot` an den echten Roboter gesendet. Im Modus `both` werden beide Wege parallel verwendet.
- **Validierung und Begrenzung** Gelenkwinkel werden auf feste Grenzwerte begrenzt. Auch die Geschwindigkeit wird in einen erlaubten Bereich gebracht.
- **Verwaltung des Simulationszustands** Für den virtuellen Betrieb werden Gelenkwinkel intern in `_sim_angles` gespeichert, damit `get_angles()` auch ohne reale Hardware einen sinnvollen Zustand liefern kann.
- **Kapselung der Herstellerbibliothek** `pymycobot` bleibt vollständig hinter dem Controller verborgen. Der Benutzer arbeitet nur mit der `Robot`-Klasse und nicht direkt mit der Hersteller-API.

6.3.2. Whitebox `World`

Die Klasse `World` ist der zweite zentrale Zugangspunkt auf Python-Seite. Sie ist bewusst unabhängig

von der Robotersteuerung.



Aufgaben der World-Komponente

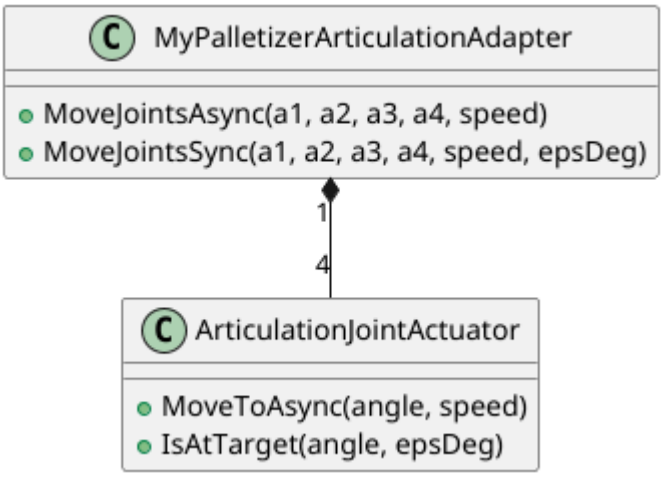
`World` kapselt alle Befehle, die sich auf die Simulationswelt beziehen. Dazu gehören:

- Erzeugen neuer Objekte
- Löschen vorhandener Objekte
- Zurücksetzen der Szene
- Laden von vordefinierten Presets

Die Trennung von `World` und `Robot` sorgt dafür, dass Weltänderungen die eigentliche Robotersteuerung nicht unnötig vermischen.

6.3.3. Whitebox `MyPalletizerArticulationAdapter`

Innerhalb der Unity-Simulation übernimmt der `MyPalletizerArticulationAdapter` die Weitergabe von Gelenkbefehlen an die einzelnen Gelenkaktuatoren.



Aufgaben des Adapters

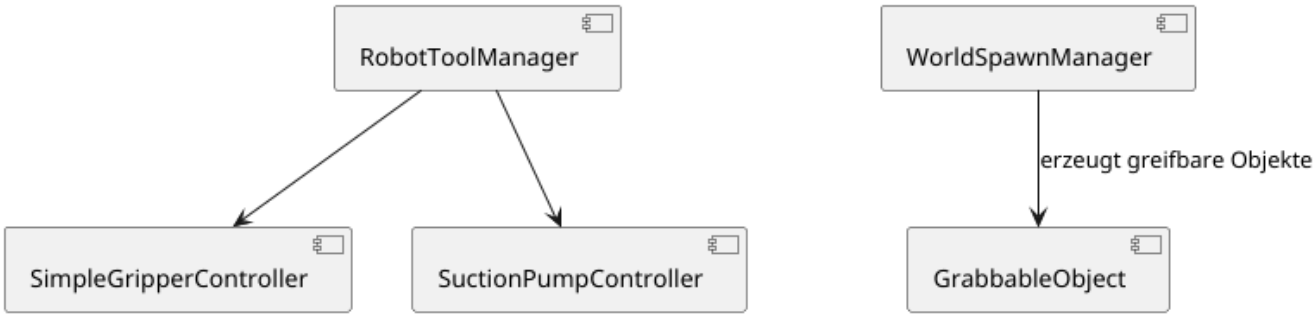
Der Adapter verteilt die empfangenen Zielwinkel auf vier Gelenkaktuatoren. Dabei gibt es zwei Ausführungsarten:

- **asynchron** über `MoveJointsAsync(…)`
- **synchron** über `MoveJointsSync(…)`, wobei gewartet wird, bis alle Gelenke ihr Ziel erreicht haben

Damit bildet der Adapter die Gelenkbewegung des Roboters in Unity auf einer klaren, gekapselten Ebene ab.

6.3.4. Whitebox Werkzeug- und Weltverwaltung in Unity

Neben den Gelenken besitzt die Unity-Seite weitere spezialisierte Komponenten für Werkzeuge und Weltobjekte.



Aufgaben dieser Komponenten

- **RobotToolManager** Koordiniert Werkzeugwechsel und Werkzeugbefehle innerhalb der Simulation.
- **SimpleGripperController** Greift Objekte innerhalb eines Radius, hält sie fest und gibt sie wieder frei.
- **SuctionPumpController** Aktiviert bzw. deaktiviert die Pumpe und bindet das nächstgelegene greifbare Objekt an den Roboter.
- **WorldSpawnManager** Erstellt Objekte anhand der empfangenen Weltnachrichten und ergänzt

dabei benötigte Komponenten wie `Rigidbody` und `GrabbableObject`.

6.4. Wichtige Entwurfsentscheidungen in der Bausteinsicht

Die Bausteinsicht zeigt drei wichtige Strukturentscheidungen des Systems:

1. **Trennung von Robotersteuerung und Weltverwaltung** Auf Python-Seite (`Robot` und `World`) sowie auf Unity-Seite (`UnityUdpBridge` und `WorldUdpBridge`) sind diese Bereiche getrennt.
2. **Kapselung externer Abhängigkeiten** Die Herstellerbibliothek `pymycobot` wird nicht direkt an den Benutzer weitergegeben, sondern vollständig über `MyPalletizerController` und `Robot` gekapselt.
3. **Symmetrische Struktur zwischen Python und Unity** Auf beiden Seiten gibt es eine getrennte Verarbeitung von Roboter- und Weltbefehlen. Das vereinfacht die Architektur und macht die Kommunikation klarer.

7. Runtime View

Die Laufzeitsicht zeigt, wie die einzelnen Bausteine des Systems zur Laufzeit zusammenarbeiten. Dabei ist wichtig, dass das System je nach Betriebsmodus (`virtual`, `real`, `both`) unterschiedliche Ausführungspfade verwendet. Die Python-Seite übernimmt die Orchestrierung, während die eigentliche Ausführung entweder in der Unity-Simulation, auf dem realen Roboter oder auf beiden Systemen gleichzeitig erfolgt.

7.1. Überblick

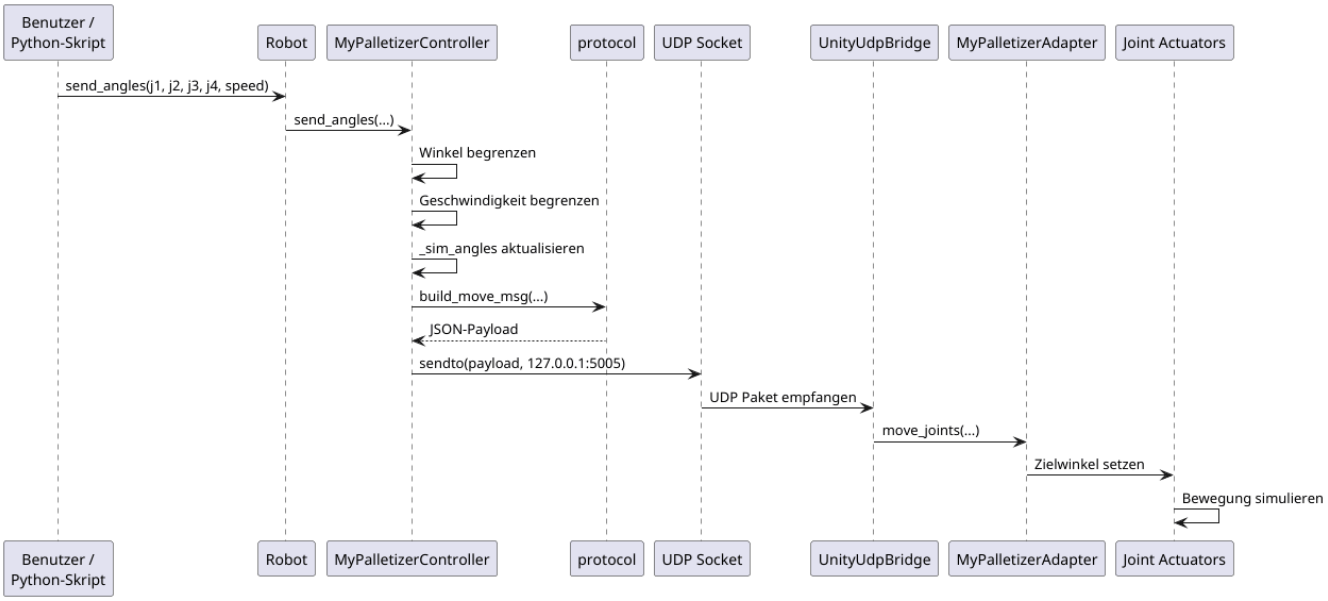
Die wichtigsten Laufzeitszenarien des Systems sind:

- asynchrone Roboterbewegung in der Simulation
- Roboterbewegung auf realer Hardware
- gleichzeitige Ausführung in Simulation und Hardware
- Verwaltung der Simulationswelt

Diese Szenarien werden in den folgenden Abschnitten beschrieben.

7.2. Szenario 1: Asynchrone Gelenkbewegung in der Simulation

Dieses Szenario beschreibt den typischen Ablauf, wenn der Benutzer einen Bewegungsbefehl im Modus `virtual` ausführt. Dabei wird ein neuer Soll-Zustand für alle vier Gelenke an die Unity-Simulation gesendet, ohne dass auf eine Rückmeldung gewartet wird.



Ablauf:

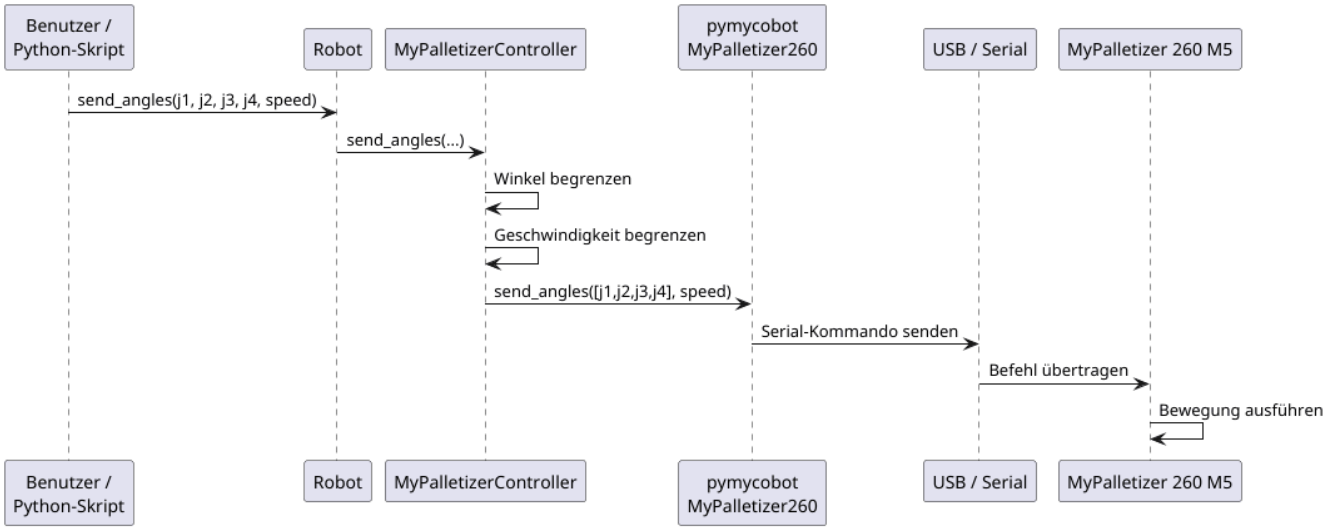
1. Der Benutzer ruft `send_angles(⋯)` über die `Robot`-API auf.
2. `Robot` delegiert den Befehl an den `MyPalletizerController`.

3. Der Controller prüft die Werte, begrenzt Winkel und Geschwindigkeit und speichert die Zielwerte intern als Simulationszustand.
4. Anschliessend wird über das `protocol`-Modul eine JSON-Nachricht erzeugt.
5. Diese Nachricht wird per UDP an Port `5005` gesendet.
6. In Unity empfängt `UnityUdpBridge` das Paket und gibt die Daten an den Adapter weiter.
7. Der Adapter steuert die simulierten Gelenke an, wodurch die Bewegung in Unity sichtbar wird.

Dieses Szenario ist der Standardfall für flüssige Bewegungen in der Simulation.

7.3. Szenario 2: Bewegung auf realer Hardware

Dieses Szenario beschreibt den Ablauf im Modus `real`. Im Unterschied zur Simulation wird der Befehl nicht per UDP gesendet, sondern über die Herstellerbibliothek `pymycobot` an den realen Roboter weitergegeben.



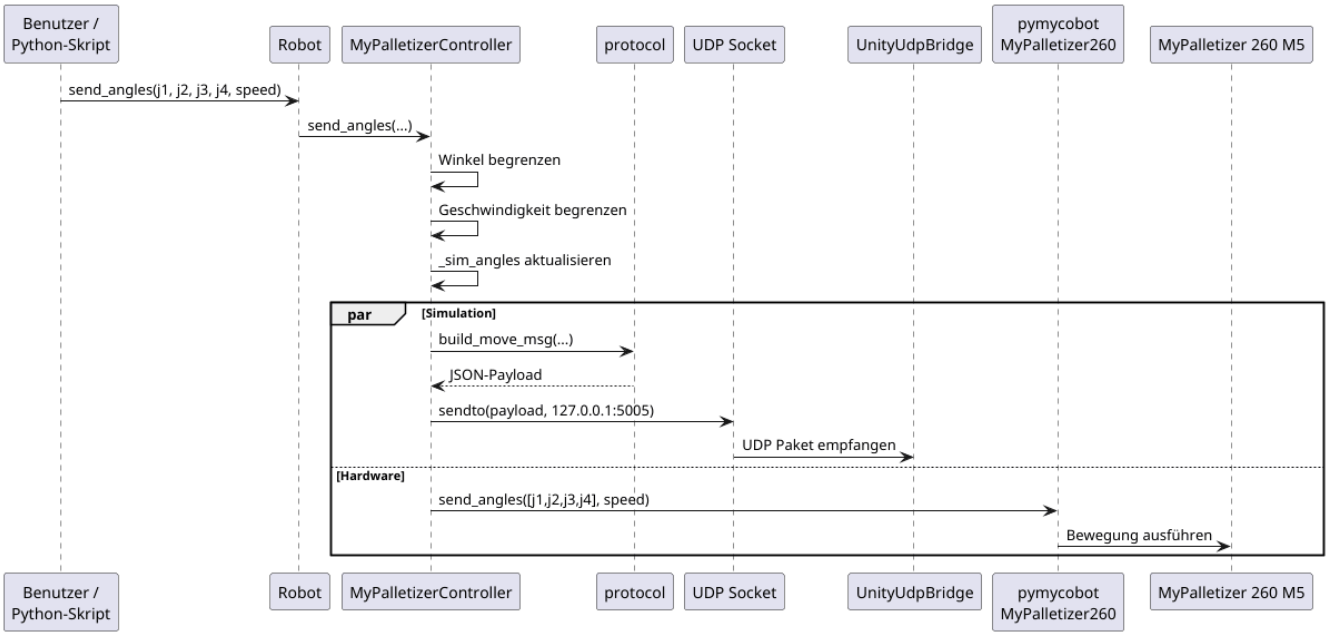
Ablauf:

1. Der Benutzer verwendet dieselbe API wie im Simulationsbetrieb.
2. `Robot` delegiert den Befehl an den `MyPalletizerController`.
3. Der Controller validiert die Eingaben.
4. Im Modus `real` wird der Befehl direkt an `pymycobot` weitergegeben.
5. Die Herstellerbibliothek sendet das Kommando über die serielle Verbindung an den realen Roboter.
6. Der Roboter führt die Bewegung physisch aus.

Dieses Szenario zeigt, dass dieselbe Benutzer-API auch für reale Hardware verwendet werden kann.

7.4. Szenario 3: Gleichzeitige Ausführung in Simulation und Hardware

Im Modus `both` wird derselbe Befehl gleichzeitig an die Simulation und an den realen Roboter gesendet. Dies ist die eigentliche Digital-Twin-Funktion des Systems.



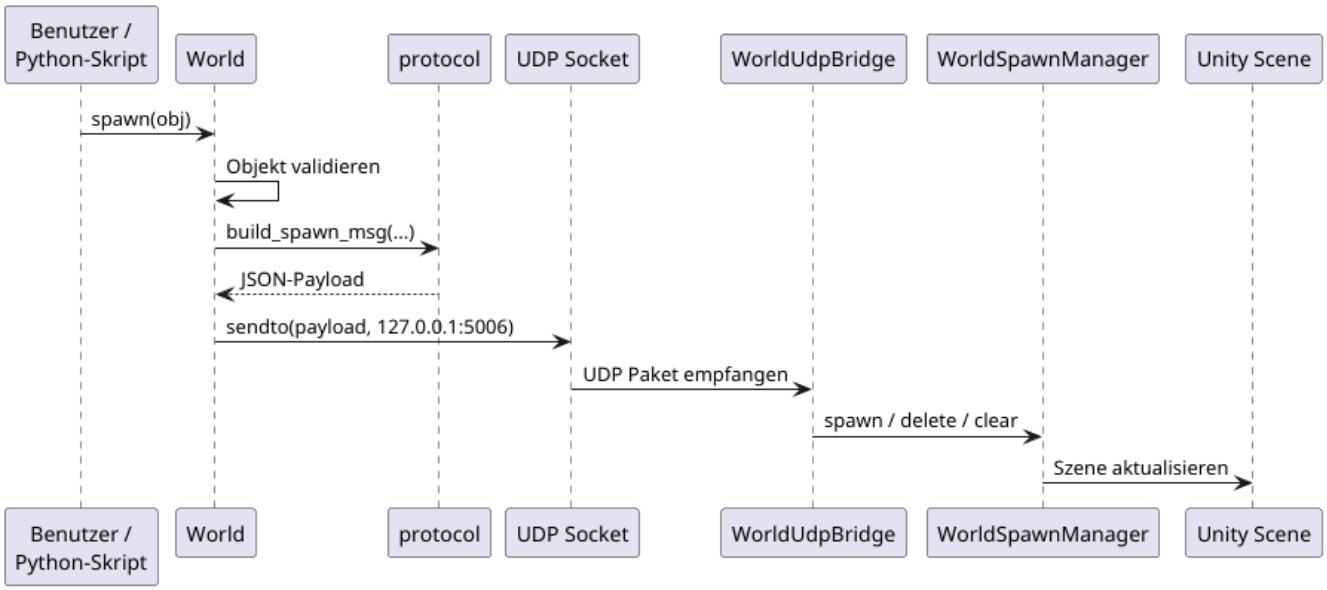
Ablauf:

1. Der Benutzer verwendet erneut dieselbe API.
2. Der `MyPalletizerController` erkennt, dass der Modus `both` aktiv ist.
3. Der Befehl wird parallel
   - an die Unity-Simulation per UDP und
   - an die Hardware über `pymycobot` weitergegeben.
4. Die Bewegung läuft dadurch gleichzeitig in der Simulation und auf dem realen Roboter.

Dieser Modus eignet sich besonders für Demonstrationen und für den direkten Vergleich zwischen Simulation und realem Verhalten.

7.5. Szenario 4: Verwaltung der Simulationswelt

Neben der Robotersteuerung unterstützt das System auch Änderungen an der Simulationswelt.



Ablauf:

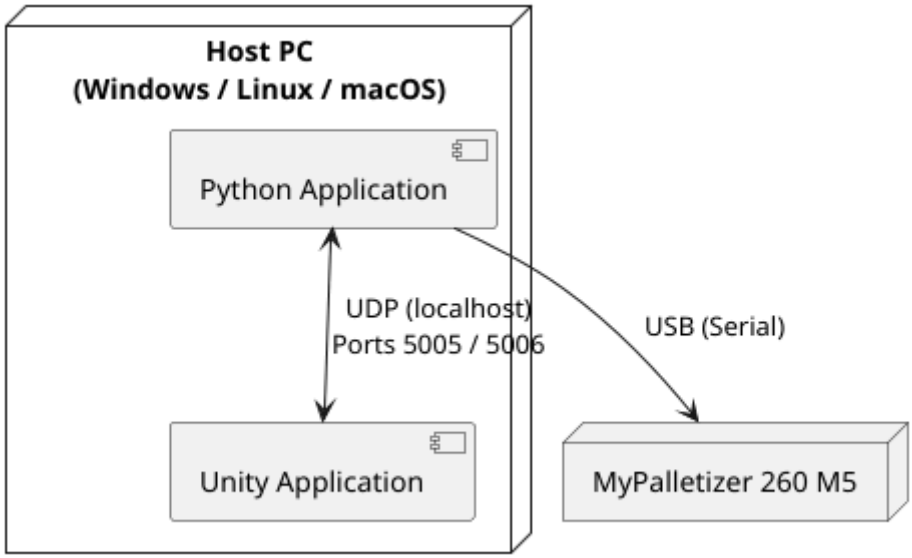
1. Der Benutzer erzeugt ein Objekt und ruft `world.spawn(obj)` auf.
2. `World` prüft das Objekt und erzeugt eine passende JSON-Nachricht.
3. Die Nachricht wird per UDP an Port `5006` gesendet.
4. `WorldUdpBridge` empfängt die Nachricht und gibt sie an den `WorldSpawnManager` weiter.
5. Dieser aktualisiert die Unity-Szene und erzeugt, löscht oder leert Objekte.

Die Trennung zwischen Roboter- und Weltkanal ist wichtig, damit häufige Bewegungsbefehle und seltenere Weltänderungen unabhängig voneinander verarbeitet werden können.

8. Deployment View

Die Verteilungssicht beschreibt die physische Infrastruktur und die Softwareumgebung, in der das System betrieben wird.

Das System ist für den Betrieb auf einem normalen Rechner ausgelegt. Die Python-Anwendung und die Unity-Simulation laufen dabei in der Regel auf demselben Host. Optional kann zusätzlich der reale MyPalletizer 260 M5 über USB angeschlossen werden.



8.1. Host-PC

Der Host-PC benötigt folgende Softwarekomponenten:

- **Python Runtime (ab Version 3.12)** Erforderlich für die Ausführung der Steuerlogik und der Python-API.
- **Unity-Anwendung** Enthält die Simulation, die Visualisierung und die Empfängerlogik für die UDP-Kommunikation.
- **USB-Seriell-Verbindung** Wird benötigt, wenn der reale Roboter verwendet werden soll.

8.2. Bereitstellung

Die Software wird über eine CI/CD-Pipeline automatisiert gebaut. Dabei werden für die unterstützten Betriebssysteme jeweils Artefakte erstellt.

Dadurch kann der Benutzer die passende Version für sein Betriebssystem direkt herunterladen und starten, ohne die Anwendung selbst bauen zu müssen. Dies vereinfacht die Nutzung erheblich und ermöglicht einen schnellen Einstieg, insbesondere für Anwender die keine Unity-Vorkenntnisse haben.

Falls der Benutzer die Simulation anpassen oder erweitern möchte, bleibt das Unity-Projekt zugänglich. Es kann mit einer kostenlosen Unity-Version geöffnet und bearbeitet werden um die Simulation nach eigenen Bedürfnissen zu verändern.

8.3. MyPalletizer 260 M5

Der reale Roboter ist ein optionaler Bestandteil der Laufzeitumgebung. Er wird dann benötigt, wenn das System im Modus `real` oder `both` verwendet wird.

Für den Betrieb sind insbesondere folgende Punkte relevant:

- Verbindung über USB / Serial
- passende Firmware auf dem Roboter
- externe Stromversorgung für den zuverlässigen Betrieb

8.4. Betriebsarten

Das System kann in drei Varianten betrieben werden:

- **virtual** – nur Simulation
- **real** – nur realer Roboter
- **both** – Simulation und realer Roboter gleichzeitig

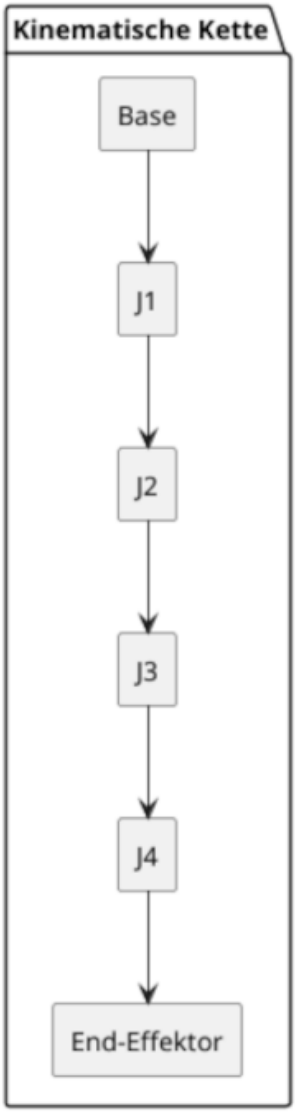
Diese Flexibilität ermöglicht es, das System an die jeweiligen Bedürfnisse und Ressourcen anzupassen. So kann beispielsweise auch ohne Zugriff auf die Hardware die Entwicklung und das Testen von Bewegungsabläufen in der Simulation erfolgen.

9. Cross-cutting Concepts

In diesem Kapitel werden Konzepte beschrieben, die in mehreren Teilen des Systems verwendet werden.

9.1. Simulationskonzept

Die Simulation des Roboters wird in Unity umgesetzt. Dabei besteht der Roboter aus mehreren miteinander verbundenen Gelenken, sodass die Untergeliederten Bauteile (z.B. J3) sich mitbewegen, wenn das übergeordnete Gelenk (z.B. J2) bewegt wird.

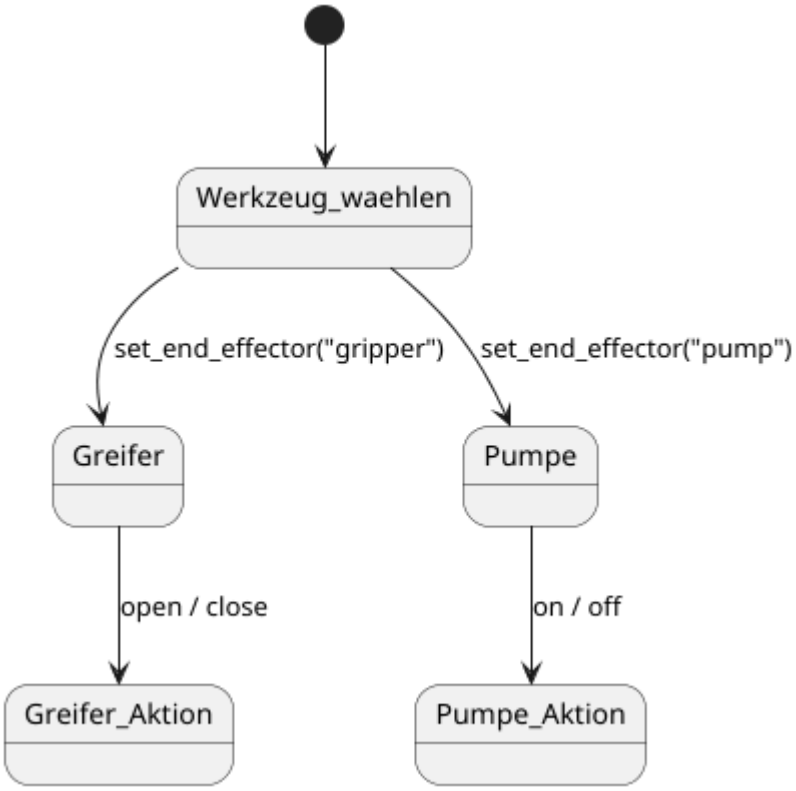


Die Trennung in einzelne Gelenke ermöglicht es, Bewegungsbefehle aus der Python-Anwendung direkt auf die simulierte Roboterstruktur abzubilden. Dadurch können Zielwinkel für die einzelnen Achsen unabhängig verarbeitet und in der Simulation dargestellt werden.

9.2. End-Effektor-Konzept

Der Roboter unterstützt unterschiedliche Werkzeuge, insbesondere Greifer und Pumpe. Diese Werkzeuge können sowohl über die Python-API ausgewählt als auch in der Simulation

nachgebildet werden.



Dieses Konzept sorgt dafür, dass verschiedene Werkzeugarten über dieselbe Schnittstelle angesprochen werden können.

9.3. Objekt- und Szenarienkonzept

Für die Simulationswelt werden einfache Objekte wie Boxen, Kugeln oder Zylinder verwendet. Diese können über die Python-Seite erstellt und in Unity dargestellt werden.

Zusätzlich unterstützt das System vordefinierte Szenarien (**Presets**), welches ein einfacherer Einstieg in die Nutzung ermöglicht.

Objekte werden dabei über ihren Namen angesprochen. So können sie später gezielt gelöscht oder verändert werden.

9.4. Fehlerbehandlung und Validierung

Ein weiteres wichtiges Konzept ist die Validierung von Eingaben vor der Ausführung.

Dazu gehören unter anderem:

- Begrenzung von Gelenkwinkeln
- Begrenzung der Geschwindigkeit
- Prüfung von Objektparametern
- Fehlerbehandlung bei ungültiger Konfiguration

Dadurch werden fehlerhafte Befehle möglichst früh erkannt und unzulässige Zustände vermieden.

9.5. Testkonzept

Das Testkonzept umfasst sowhol Unit-Tests für die Python-API als auch für die Unity-Komponenten. Dabei werden typische Anwendungsfälle, Grenzfälle und Fehlerfälle abgedeckt, um die Stabilität und Zuverlässigkeit des Systems sicherzustellen.

Zudem wurden auch Systemtests implementiert, die Manuell ausgeführt wurden. Diese Tests decken auch die Interaktion zwischen Python und Unity ab, um sicherzustellen, dass die Kommunikation korrekt funktioniert und die Simulation wie erwartet reagiert. Dabei wurden diese im Github-Wiki dokumentiert, damit sie auch von anderen Benutzern nachvollzogen werden können LINK.

10. Architecture Decisions

Die folgenden Architectural Decision Records (ADR) beschreiben wichtige Architekturentscheidungen des Projekts.

10.1. ADR 1: Einheitliche Python-API für Simulation und Hardware

- **Kontext**: Der Benutzer soll nicht zwischen verschiedenen APIs für Simulation und reale Hardware unterscheiden müssen.
- **Entscheidung**: Die Klassen `Robot` und `World` bilden die zentrale Benutzerschnittstelle. Intern wird die eigentliche Ausführung an den `MyPalletizerController` oder an die Weltkommunikation delegiert.
- **Konsequenz**: Die Benutzung des Systems wird einfacher und konsistenter. Gleichzeitig übernimmt die Python-Seite mehr Verantwortung, da sie Validierung, Moduswahl und Weiterleitung der Befehle kapselt.

10.2. ADR 2: Kapselung der Herstellerbibliothek `pymycobot`

- **Kontext**: Der reale Roboter soll unterstützt werden, ohne dass der Benutzer direkt von der Herstellerbibliothek abhängig ist.
- **Entscheidung**: Die Bibliothek `pymycobot` wird nur intern im `MyPalletizerController` verwendet. Der Benutzer arbeitet ausschliesslich mit der eigenen `Robot`-Schnittstelle.
- **Konsequenz**: Externe Abhängigkeiten bleiben gekapselt und die API nach aussen bleibt stabiler. Dafür entsteht eine zusätzliche Abstraktionsschicht, die gepflegt werden muss.

10.3. ADR 3: Trennung von Simulation und realer Hardware

- **Kontext**: Dasselbe System soll sowohl mit der Simulation als auch mit dem realen Roboter funktionieren.
- **Entscheidung**: Die Simulation wird über UDP angesprochen, die reale Hardware über `pymycobot` und die serielle Verbindung. Die Auswahl erfolgt intern über die Modi `virtual`, `real` und `both`.
- **Konsequenz**: Dieselbe Python-API kann für Simulation, reale Hardware oder beide gleichzeitig verwendet werden. Dadurch bleibt die Benutzerschnittstelle einfach, obwohl intern unterschiedliche Kommunikationswege verwendet werden.

10.4. ADR 4: Trennung von Roboter- und Weltkommunikation

- **Kontext**: Das System verarbeitet zwei unterschiedliche Arten von Befehlen: Roboterbefehle und Befehle zur Verwaltung der Simulationswelt.
- **Entscheidung**: Diese beiden Bereiche werden über getrennte UDP-Kanäle umgesetzt. Port `5005` wird für die Robotersteuerung verwendet, Port `5006` für Weltbefehle.
- **Konsequenz**: Die Kommunikation bleibt klar getrennt. Häufige Bewegungsbefehle des Roboters werden nicht mit Weltbefehlen wie `spawn`, `delete` oder `clear` vermischt.

10.5. ADR 5: Verwendung von Presets für wiederholbare Szenarien

- **Kontext**: Typische Testszenarien sollen schnell geladen und reproduzierbar ausgeführt werden können.
- **Entscheidung**: Vordefinierte Szenarien werden als Presets auf Python-Seite bereitgestellt und über `World.load_preset(…)` geladen.
- **Konsequenz**: Tests und Demonstrationen werden einfacher und konsistenter. Gleichzeitig müssen Presets gepflegt werden, wenn sich die Simulationswelt ändert.

10.6. ADR 6: Verwendung von Articulation Bodies in Unity

- **Kontext**: Der Roboterarm besteht aus mehreren Gelenken, die in der Simulation stabil und nachvollziehbar bewegt werden müssen.
- **Entscheidung**: Für die Simulation der Gelenkkette werden in Unity `ArticulationBody` -Komponenten verwendet.
- **Konsequenz**: Die Abbildung des Roboterarms in Unity wird robuster und besser für zusammenhängende Gelenkstrukturen geeignet. Gleichzeitig wird die Konfiguration in Unity etwas aufwendiger.

11. Quality Requirements

Die Qualitätsanforderungen machen die wichtigsten Ziele des Systems anhand konkreter Szenarien messbar.

11.1. Quality Requirements Overview

Für dieses System sind vor allem eine schnelle Reaktion, eine einfache Nutzung sowie ein robuster Betrieb wichtig. Da das System sowohl in der Lehre als auch für Tests verwendet wird, spielen zudem Portabilität und Wartbarkeit eine wichtige Rolle.

Qualitätsmerkmal	Priorität
Performance	Sehr hoch
Bedienbarkeit	Hoch
Portabilität	Hoch
Robustheit	Hoch
Wartbarkeit	Mittel

11.2. Quality Scenarios

ID	Qualitätsziel	Szenario
QS-1	Performance	Ein Benutzer ruft `send_angles(…)` auf. Die Simulation soll innerhalb kurzer Zeit sichtbar auf den Befehl reagieren. Ziel ist, dass die Bewegung in Unity innerhalb von spätestens 50 ms beginnt.
QS-2	Portabilität	Ein Benutzer wechselt von einem Windows-System auf macOS oder Linux. Dasselbe Python-Skript soll ohne Änderungen weiterverwendet werden können, sofern die benötigten Abhängigkeiten installiert sind.
QS-3	Robustheit	Die Unity-Simulation wird während der Ausführung eines Python-Skripts beendet. Das Python-Skript soll dadurch nicht blockieren oder abstürzen.
QS-4	Eingabevalidierung	Ein Benutzer sendet einen Gelenkwinkel ausserhalb des erlaubten Bereichs. Das System muss diesen Wert vor der Weitergabe an Simulation oder Hardware auf den zulässigen Bereich begrenzen.

ID	Qualitätsziel	Szenario
QS-5	Bedienbarkeit	Ein neuer Benutzer installiert die Software und führt ein Beispielskript aus. Innerhalb von 15 Minuten soll er in der Lage sein, eine Bewegung in der Simulation erfolgreich zu starten.
QS-6	Wartbarkeit	Eine neue Nachricht für die Simulation soll ergänzt werden können, ohne bestehende Roboter- und Weltbefehle grundlegend umzubauen.

12. Risks and Technical Debts

Dieses Kapitel beschreibt mögliche Risiken sowie technische Kompromisse, die im aktuellen System bestehen.

12.1. Architektur-Risiken

Risiko	Erläuterung und Minderung
UDP ohne garantierte Zustellung	Da UDP keine garantierte Zustellung bietet, können einzelne Pakete verloren gehen. **Minderung**: Das Protokoll überträgt jeweils den vollständigen Soll-Zustand, sodass ein späteres Paket einen verlorenen Zustand wieder korrigieren kann.
Unterschiedliches Verhalten von Simulation und Hardware	Im Modus `both` kann es vorkommen, dass Simulation und realer Roboter nicht exakt gleich schnell reagieren. **Minderung**: Beide Systeme verwenden dieselben Zielwerte und denselben `speed` -Parameter. Für eine genauere Synchronisation wäre in Zukunft ein Rückkanal mit Ist-Werten sinnvoll.
Port-Konflikte	Die verwendeten UDP-Ports `5005` und `5006` könnten auf einem System bereits belegt sein. **Minderung**: Die Ports sind konfigurierbar und können bei Bedarf angepasst werden.

12.2. Implementierungs-Risiken und technische Schulden

- **Abhängigkeit von Unity** Die Simulation hängt von Unity und dessen Physikverhalten ab. Änderungen an der Unity-Version können Einfluss auf das Verhalten der Simulation haben.
- **Open-Loop-Steuerung** Die Python-Seite sendet Befehle, ohne aktiv zu prüfen, ob diese in der Simulation oder auf der Hardware exakt wie erwartet ausgeführt wurden. Dies reduziert die Komplexität, bedeutet aber auch, dass kein vollständiges Feedback-System vorhanden ist.
- **Begrenzte Modellgenauigkeit** Die Simulation bildet den realen Roboter nur näherungsweise ab. Werte wie Bewegung, Reibung oder Interaktionen mit Objekten sind ausreichend für Tests und Demonstrationen, aber nicht für hochpräzise physikalische Analysen.
- **Abhängigkeit von `pymycobot`** Der Modus `real` hängt von der externen Herstellerbibliothek `pymycobot` ab. Änderungen oder Inkompatibilitäten in dieser Bibliothek können direkte Auswirkungen auf die Hardwareansteuerung haben. Eine mögliche spätere Verbesserung wäre eine noch stärkere Entkopplung durch eine eigene Hardware-Abstraktionsschicht.
- **Kein Rückkanal aus der Simulation** Die Kommunikation zur Simulation erfolgt aktuell hauptsächlich in eine Richtung. Dadurch erhält die Python-Seite keine detaillierten Laufzeitinformationen aus Unity zurück. Für zukünftige Erweiterungen könnte ein Rückkanal hilfreich sein, zum Beispiel für Statusinformationen oder Fehlerfälle.

13. Glossary

Term	Definition
Digital Twin	*Ein digitaler Zwilling ist ein virtuelles Abbild eines physischen Objekts (hier: MyPalletizer 260 M5), das dessen Verhalten und Zustand in Echtzeit simuliert.*
Articulation Body	*Eine spezielle Komponente in Unity zur physikalischen Simulation von kinematischen Ketten (Robotern), die für hohe Stabilität bei Gelenkbewegungen sorgt.*
UDP	*User Datagram Protocol. Ein verbindungsloses Netzwerkprotokoll, das für seine geringe Latenz bekannt ist.*
JSON	*JavaScript Object Notation. Ein kompaktes Datenformat zum Austausch von strukturierten Daten zwischen Python und Unity.*
pymycobot	*Die offizielle Python-Library des Herstellers Elephant Robotics zur Ansteuerung der MyRobot-Serie.*
APPLAI	*Applied Artificial Intelligence: Das Modul an der HSLU, in dem dieses Projekt entwickelt wurde.*
HSLU	*Hochschule Luzern*

HSLU Hochschule Luzern